

어디서 계산하고, 무엇을 기억하는가

8개 주제의 갱신 흐름과 방법 사이의 관계를 연결한 학습 페이지

무엇을 계산하고, 무엇을 기억할까?

두 방법의 필요한 연산과 상태를 나란히 봅니다. 문제 계열과 한 단계의 비용이 달라 수렴 속도 순위표로 쓰지 않습니다.

첫 번째 방법

Conjugate gradient

Conjugate gradient

필요한 정보

SPD 선형계와 행렬-벡터 곱

기억하는 상태

보정점, 잔차, 탐색 방향

한 갱신의 핵심 작업

방향 계산과 실제 잔차 재계산에 행렬-벡터 곱 사용

위는 정확한 산술의 방향 구조입니다. 실제 코드는 크기를 조정된 보정 방정식을 풀고 실제 잔차로 종료를 확인합니다.

[질문·가정·실제 곡선 읽기](#)

두 번째 방법

Proximal point

Proximal point

필요한 정보

정확히 풀 수 있는 SPD 이차 하위 문제

기억하는 상태

현재점 x 와 고정된 Q, c

한 갱신의 핵심 작업

갱신마다 선형계 풀이; 기울기 한 번과 같은 비용이 아닙니다.

전체 이차함수를 prox 안에서 풉니다. FISTA는 smooth 항을 선형화하고 nonsmooth 항에 prox를 적용합니다.

[질문·가정·실제 곡선 읽기](#)

1 · 외삽점에서 기울기

$$z = y_k - \nabla f(y_k) / L$$



2 · nonsmooth 항의 prox

$$x_{k+1} = \text{prox}_\lambda(g/L)(z); \quad L1: \text{soft}(z, \lambda/L)$$



3 · 가속 계수 갱신

$$t_{k+1} = (1 + \sqrt{1 + 4t_k^2}) / 2$$



4 · 다음 외삽점

$$y_{k+1} = x_{k+1} + ((t_k - 1) / t_{k+1})(x_{k+1} - x_k)$$

실제 HTML 브라우저 캡처. 위: CG와 이차 proximal point의 연산·상태 비교. 아래: FISTA의 기호적 4단계 갱신 흐름. 이 그림은 계산 구조를 설명하며, 수치 관측은 각 주제의 기존 검증된 곡선에서 따로 확인합니다.

수식 한 줄에서 이해를 멈추지 않도록

같이 보이는 한 반복의 차이

CG는 방향 계산과 실제 잔차 확인에 행렬-벡터 곱을 사용합니다. 이차 proximal point는 매 갱신에서 선형계를 풉니다. 비교 카드는 필요한 정보와 연산을 설명하며 속도 순위를 만들지 않습니다.

역사·구현·관측을 구분합니다.

Frank-Wolfe의 1956년 원 방법과 Jaggi의 2013년 분석을 각각 표시합니다. Nesterov의 역사적 이름과 실제 smooth-FISTA 갱신식도 구분합니다. 방법 간 연결은 수학적 관계이며 역사적 파생을 단정하지 않습니다.

검증: 615개 테스트·Ruff-지정 오류 주입 4종·빌드 통과. 1440px와 390px에서 전체 비교 선택, 중복 선택 처리, 키보드, 검색 후 이동, 언어 전환, JSON 다운로드, 단일 주제 내보내기를 확인했습니다. HTTP 요청과 JavaScript 오류는 없었습니다.

JavaScript 없이도 8개 비교 카드와 모든 흐름도·주제·수치 곡선을 읽을 수 있습니다. JSON에 기호적 설명을 별도 필드로 담고 기존 수치 결과와 검사 의미를 유지했습니다.

PR: #54 · #52 위의 미병합 변경

CI: 8개 작업 성공; wheel-sdist 설치 확인

head: 3ca00c0499fcf180184c3a659acb364009169f08

논문의 모든 실험을 재현하거나 방법의 보편적인 우열을 증명하는 자료가 아닙니다. 공개 릴리스는 v0.5.0이며, 이번 개선은 개발 브랜치에 있습니다.

출처: Frank-Wolfe (1956) · Jaggi (2013). 각 갱신식·보장·검사 대응: docs/SOURCE_MAP.md. 읽기와 사용법: docs/LEARNING_WORKFLOWS.md.

관련된 질문으로 이동

“ $g=0$ 이면?”, “외삽점에서 기울기를 계산하면?” 같은 질문을 누르면 해당 주제로 이동합니다. 검색으로 가려진 주제도 다시 표시합니다. 한 주제만 내보낸 파일에는 다른 주제를 생성할 실행 명령을 제공합니다.

무엇을 계산하고, 무엇을 기억할까?

두 방법의 필요한 연산과 상태를 나란히 봅니다. 문제 계열과 한 단계의 비용이 달라 수렴 속도 순위표로 쓰지 않습니다.

첫 번째 방법

Conjugate gradient

두 번째 방법

Proximal point

Conjugate gradient

필요한 정보

SPD 선형계와 행렬-벡터 곱

가역하는 상태

보장점, 잔차, 탐색 방향

한 갱신의 핵심 작업

방향 계산과 실제 잔차 재계산에 행렬-벡터 곱 사용

위는 정확한 산술의 방향 구조입니다. 실제 코드는 크기를 조정된 보장 방정식을 풀고 실제 잔차로 종료 확인합니다.

[질문·가정·실제 곡선 읽기](#)

Proximal point

필요한 정보

정확히 풀 수 있는 SPD 이차 하위 문제

가역하는 상태

현재점 x 와 고정된 Q, c

한 갱신의 핵심 작업

갱신마다 선형계 풀이; 기울기 한 번과 같은 비용이 아닙니다.

전체 이차함수를 prox 안에서 풉니다. FISTA는 smooth 항을 선형화하고 nonsmooth 항에 prox를 적용합니다.

[질문·가정·실제 곡선 읽기](#)

390px 화면에서는 비교 카드를 세로로 배치합니다.